

Corso di Laurea in Ingegneria e Scienze Informatiche

Fancy Title

Tesi di laurea in:
NOME DEL CORSO DEL RELATORE

Relatore

Prof. Supervisor Here

Candidato

Candidate Name

Abstract

Opzionale. Poche righe al massimo.

Contents

Abstract	iii
1 Introduzione	1
1.1 Contesto e motivazione	1
1.2 Obiettivi del progetto	1
1.3 Struttura della tesi	1
2 Background	3
2.1 Sviluppo di applicazioni full stack monolinguaggio	3
2.1.1 Panoramica delle soluzioni esistenti e loro limiti	4
2.1.2 La portabilità e integrazione con librerie native	5
2.2 Kotlin e framework multitarget come risposta al gap	6
2.2.1 Kotlin Multiplatform	6
2.2.2 Interoperabilità con C tramite Cinterop	7
2.3 Lo standard FMI e il formato FMU	8
2.3.1 Functional Mock-up Interface (FMI 2.0)	8
2.3.2 Struttura di un file FMU e modalità operative	9
3 Analisi	11
3.1 Requisiti	11
3.2 Modello del dominio	12
3.2.1 Vincoli	14
4 Design	15
4.1 Architettura	15
4.2 Design di dettaglio	16
4.2.1 Interfacciamento con librerie native	17
4.2.2 Backend multiplatforma	19
4.2.3 Frontend multiplatforma	20

5	Implementazione	23
5.1	Binding C/Kotlin e configurazione del linker per piattaforma	24
5.2	Gestione del ciclo di vita dell'FMU	25
5.3	Estrazione dei metadati e gestione delle variabili	25
5.4	Esecuzione della simulazione	25
5.5	Ricompilazione degli FMU su macOS ARM64	25
5.6	Strumenti di processo: Gradle, CI/CD e pipeline multiplatforma .	25
6	Valutazione	27
6.1	Test unitari e di integrazione	27
6.2	Copertura multiplatforma nella CI	27
7	Conclusioni e Sviluppi Futuri	29
7.1	Limitazioni	29
7.2	Sviluppi futuri	29
7.3	Conclusioni	29
		31
	Bibliography	31

List of Figures

3.1	Entità dell'applicativo	13
4.1	Diagramma rappresentante relazione tra i moduli dell'applicativo. .	16
4.2	Design del modulo fmu-kt	18
4.3	Design del modulo backend	20

LIST OF FIGURES

List of Listings

LIST OF LISTINGS

Chapter 1

Introduzione

1.1 Contesto e motivazione

1.2 Obiettivi del progetto

1.3 Struttura della tesi

Chapter 2

Background

2.1 Sviluppo di applicazioni full stack monolinguaggio

Prima di addentrarci nella descrizione del background del progetto è importante definire in maniera corretta cosa sono le applicazioni **full stack** e le differenze che emergono tra lo sviluppo di esse e quello di applicazioni più tradizionali. Le componenti delle applicazioni vengono divise in due macrocategorie

- **Frontend:** in questa sezione dell'applicativo sono gestiti tutti gli aspetti inerenti all'interfaccia utente: sia quelli legati all'aspetto grafico che quelli legati alla logica di interazione con essa.
- **Backend:** in questa sezione invece sono gestiti tutti gli aspetti legati all'esecuzione delle funzionalità dell'applicazione, inclusa l'elaborazione dei dati, la loro memorizzazione e la comunicazione con altre applicazioni se necessario.

Le applicazioni *full stack* dunque, comprendono entrambe le componenti. Lo sviluppo di questo tipo di applicazioni può includere l'utilizzo di molteplici linguaggi in base alle necessità, impiegando linguaggi progettati per lo sviluppo del backend e altri per il frontend. Tuttavia questa distinzione al giorno d'oggi viene gradualmente smussata, infatti diversi linguaggi inizialmente progettati per

l'utilizzo in una sola area, vengono poi estesi anche all'altra. L'esempio più rilevante e dominante al giorno d'oggi è **JavaScript**, il quale inizialmente nasce come linguaggio per i browser, quindi dedicato al *frontend*, per poi evolversi oltre i confini del browser permettendone l'esecuzione anche sui server. Questo permette dunque la creazione di applicativi *full stack* monolinguaggio, una tendenza sempre più diffusa. Tra i vantaggi che essa comporta vi sono l'utilizzo di un solo linguaggio per tutto lo stack, il quale porta a una minore curva di apprendimento e una maggiore efficienza per quanto riguarda l'impiego di programmatori. Uno dei più grandi limiti dello sviluppo *full stack* è l'interazione con librerie e Application Programming Interface (API) native, infatti linguaggi come JavaScript sono noti per avere meccanismi limitati o comunque ostici per interagire con il codice nativo. In seguito verranno discusse in dettaglio le soluzioni esistenti per la programmazione di tale tipo di applicativi e i loro limiti, per poi analizzare il problema della portabilità e dell'integrazione con le librerie native.

2.1.1 Panoramica delle soluzioni esistenti e loro limiti

Come anticipato in precedenza, esistono diversi linguaggi impiegati per la programmazione di app *full stack* monolinguaggio. Il più popolare attualmente risulta essere JavaScript [Sta25], questo per via dello sviluppo della tecnologia **Node.js**; infatti questo permette di portare l'ambiente di esecuzione di JavaScript fuori dai browser web. Inoltre anche la creazione di TypeScript, una diretta evoluzione di JavaScript, contribuì alla crescita di questo tipo di linguaggi. Tutti questi fattori rendono JavaScript una delle scelte più gettonate per la creazione di software *full stack*, impiegando diversi stack come Next.js, framework sviluppato da Vercel, e lo stack MEAN, il quale consiste in Mongo DB, Express.js e Angular come framework per il backend e frontend rispettivamente, il tutto legato da Node.js. Nonostante il grande supporto in termini di librerie e framework per questo tipo di tecnologia, l'integrazione con le librerie native risulta comunque ardua per via delle peculiarità del linguaggio. Emerge dunque un limite: l'integrazione con librerie native risulta problematica, come verrà analizzato nella sezione seguente.

2.1.2 La portabilità e integrazione con librerie native

Per librerie native si intendono tutte le librerie compilate in linguaggio macchina specifico per un architettura. Queste espongono delle **ABI** (application binary interface) ovvero delle interfacce applicative che comunicano direttamente con il sistema operativo a livello di linguaggio macchina. Tra i linguaggi che vengono compilati in codice macchina figurano, C, C++, Rust e Zig; questi spesso vengono utilizzati per la realizzazione di programmi che hanno necessità di accedere direttamente all'hardware sottostante, oppure per sviluppare librerie che implementano alcuni standard. Quest'ultimo caso è il più rilevante per la tesi, infatti l'applicativo realizzato per questa tesi interagisce con dei file aderenti allo standard Functional Mock-up Interface (FMI), il quale verrà approfondito in seguito nella sezione dedicata.

Diverse delle soluzioni menzionate nella sezione precedente, richiedono la scrittura di codice collante tra il proprio applicativo e le librerie native in caso si vogliano utilizzare. Nel caso più rilevante, ovvero JavaScript, se si necessita di interagire con librerie scritte in C++, è necessario utilizzare una libreria C++ denominata **Node-API**. Tale libreria verrà poi utilizzata per creare moduli che fungono da collante tra JavaScript e la libreria nativa, infatti questo modulo sarà responsabile delle chiamate alla libreria nativa [Nod25]. Questa soluzione permette dunque l'integrazione con questo tipo di librerie, ma contraddice direttamente il principio dietro lo sviluppo *full stack monolinguaggio*, rendendo necessaria la scrittura di codice in linguaggi al di fuori di quello usato per la realizzazione dell'applicativo. Da questa analisi emerge un pattern ricorrente: le soluzioni *full stack monolinguaggio* più mature non sono state costruite attorno all'interoperabilità nativa come requisito principale. Le soluzioni legate a questi linguaggi dunque richiedono di violare il principio del monolinguaggio per interagire con le librerie native. Questo limite non dipende da aspetti implementativi risolvibili con nuove librerie, bensì una conseguenza strutturale basata sulla natura del linguaggio scelto. Dunque una reale soluzione richiederebbe un approccio diverso fin dalla scelta del linguaggio, rendendo necessario l'impiego di un linguaggio **multitarget**.

2.2 Kotlin e framework multitarget come risposta al gap

Alla luce delle limitazioni analizzate nella sezione precedente — in particolare la difficoltà di integrare librerie native C in ambienti JavaScript e la necessità di ricorrere a strati di collante eterolinguistici — questa sezione presenta Kotlin Multiplatform come risposta tecnologicamente motivata a tale gap. L'obiettivo non è descrivere la tecnologia in modo esaustivo, ma illustrare perché le sue caratteristiche architetturali si adattano al problema in esame.

2.2.1 Kotlin Multiplatform

Kotlin nasce nel 2011 come linguaggio sviluppato da JetBrains con l'obiettivo di modernizzare lo sviluppo sulla Java Virtual Machine (JVM): sintassi più concisa, sistema di tipi con null safety integrata e piena interoperabilità con Java esistente [AB⁺20]. La maturità dell'ecosistema e il supporto di JetBrains — nonché l'adozione ufficiale da parte di Google come linguaggio primario per lo sviluppo Android — ne hanno consolidato la credibilità come linguaggio di produzione.

Kotlin Multiplatform (KMP) è un'estensione del compilatore Kotlin che permette di compilare lo stesso codice sorgente verso target eterogenei: JVM, JavaScript per il browser, e codice nativo tramite backend LLVM [Jet24c]. Il punto centrale è che KMP non è un framework di astrazione che interpone uno uniformizzante tra il codice e la piattaforma: genera artefatti nativi distinti per ciascun target, mantenendo le caratteristiche di performance e interoperabilità proprie di ciascuno. Questa distinzione è rilevante rispetto ad approcci come React Native o Flutter, dove il codice condiviso viene eseguito in un runtime separato.

Il meccanismo che rende possibile la condivisione selettiva del codice è la dichiarazione `expect/actual` [Jet24a]. Nel modulo condiviso (`commonMain`) è possibile dichiarare una funzione o una classe come `expect`, specificandone la firma senza fornirne l'implementazione. Ciascun modulo specifico di piattaforma fornisce poi la corrispondente dichiarazione `actual`, con un'implementazione che può sfruttare le API native del target. Questo meccanismo consente di separare nettamente la logica di business — che risiede nel modulo comune ed è condivisa

tra tutti i target — dalle implementazioni dipendenti dalla piattaforma, senza rinunciare al controllo dei tipi a tempo di compilazione.

Dal punto di vista del gap identificato in precedenza, la caratteristica rilevante di KMP è la presenza del target **Kotlin/Native**: il compilatore produce binari nativi tramite LLVM, senza dipendenza da una JVM a tempo di esecuzione. Questo apre la possibilità di eseguire codice Kotlin su piattaforme embedded, su macOS e iOS, e in generale in qualsiasi contesto dove una JVM non è disponibile o desiderabile. Soprattutto, Kotlin/Native offre un meccanismo diretto per interfacciarsi con librerie C — aspetto che viene trattato nella sottosezione successiva.

2.2.2 Interoperabilità con C tramite Cinterop

Kotlin/Native include uno strumento dedicato all'interoperabilità con librerie C denominato `cinterop` [Jet24b]. Il suo funzionamento si basa sulla lettura degli header `.h` di una libreria C: a partire da questi, `cinterop` genera automaticamente un modulo Kotlin contenente le definizioni corrispondenti — funzioni, struct, enumerazioni e costanti — con tipi Kotlin appropriati. Lo sviluppatore configura il processo tramite un file `.def`, in cui specifica quali header includere, eventuali flag di compilazione, e parametri di mappatura dei tipi. Il tutto avviene a tempo di compilazione, producendo un modulo che può essere importato nel codice Kotlin come qualsiasi altra dipendenza.

Il confronto con Node-API, descritta nella sezione precedente, è diretto. Con Node-API l'integrazione di una libreria C in un progetto JavaScript richiedeva la scrittura manuale di codice C++ come strato di collegamento: il binding tra i tipi JavaScript e quelli C doveva essere gestito esplicitamente dallo sviluppatore, con tutti i costi di manutenzione e le insidie di type safety che ne derivano. Con `cinterop`, questo strato viene generato automaticamente dal compilatore a partire dagli header: lo sviluppatore scrive esclusivamente in Kotlin, senza mai uscire dal proprio linguaggio di riferimento. Il principio del monolinguaggio — compromesso nell'approccio Node-API — viene quindi preservato.

Vale la pena segnalare un limite architetturale rilevante: `cinterop` è disponibile esclusivamente nel target Kotlin/Native. Il codice che utilizza collegamenti generati da `cinterop` non può quindi risiedere nel modulo `commonMain` di un pro-

getto KMP, ma deve essere collocato nel modulo nativo specifico. In pratica, questo significa che le chiamate dirette alle librerie C costituiscono un'implementazione **actual** nel modulo nativo, mentre l'interfaccia esposta al codice comune viene dichiarata tramite la corrispondente dichiarazione **expect**. Non si tratta di una limitazione bloccante, ma di un vincolo di design che influenza la struttura del progetto e che verrà ripreso nella descrizione architetturale nei capitoli successivi.

2.3 Lo standard FMI e il formato FMU

La simulazione di sistemi dinamici complessi richiede spesso l'integrazione di componenti provenienti da strumenti e ambienti di sviluppo eterogenei. Lo standard FMI nasce proprio per rispondere a questa esigenza, definendo un'interfaccia comune che consente lo scambio e l'esecuzione di modelli di simulazione indipendentemente dallo strumento con cui sono stati prodotti. Questa sezione presenta lo standard e il formato archivistico che ne è l'artefatto centrale, il file Functional Mock-up Unit (FMU), con l'obiettivo di fornire il contesto necessario a comprendere le scelte progettuali descritte nei capitoli successivi.

2.3.1 Functional Mock-up Interface (FMI 2.0)

FMI è uno standard aperto che definisce un'interfaccia C e un formato di impacchettamento per lo scambio di modelli di simulazione dinamica tra strumenti diversi [Mod14]. Lo sviluppo dello standard è stato avviato da Daimler AG nell'ambito del progetto europeo ITEA2 MODELISAR con l'obiettivo di semplificare lo scambio di modelli di simulazione tra fornitori e case automobilistiche [Mod14]. La versione 1.0 è stata pubblicata nel 2010; la versione 2.0, che è quella rilevante per questo progetto, è stata rilasciata nel 2014 e ha raggiunto una diffusione molto ampia, con supporto da parte di oltre 170 strumenti di simulazione [Mod24].

L'idea centrale di FMI è la separazione netta tra la descrizione dell'interfaccia del modello — espressa in un file XML — e la sua implementazione computazionale, distribuita come libreria condivisa compilata in C. Questa separazione consente a qualsiasi strumento compatibile di importare e simulare un componente senza

conoscere i dettagli interni del modello né dipendere dallo strumento che lo ha generato.

Lo standard definisce due modalità operative distinte, che determinano come il componente viene integrato nel sistema simulante:

- **Model Exchange (ME):** l'FMU espone le equazioni del modello — tipicamente sotto forma di equazioni differenziali ordinarie — senza includere un risolutore numerico. È lo strumento importatore a fornire il risolutore e a gestire l'integrazione. Questa modalità offre maggiore controllo sulla precisione numerica ed è preferita quando il modello deve essere integrato strettamente in un ambiente di simulazione esistente.
- **Co-Simulation (CS):** l'FMU include il proprio risolutore numerico. Lo strumento importatore si limita a impostare gli ingressi, a richiedere all'FMU di avanzare di un passo temporale tramite la funzione `fmi2DoStep`, e a leggere le uscite al termine del passo $[T+21]$. Questa modalità è la più diffusa in pratica, in quanto tratta il componente come una scatola nera e minimizza le dipendenze tra simulatore e modello.

L'interfaccia C esposta dall'FMU segue una convenzione di denominazione prefissata: le funzioni hanno nomi nella forma `fmi2<NomeFunzione>`, dove il prefisso `fmi2` identifica la versione dello standard. Il ciclo di vita di un'istanza FMU prevede fasi distinte — istanziamento, inizializzazione, simulazione, terminazione — ciascuna governata da specifiche chiamate di funzione. Questo contratto è ciò che garantisce la portabilità: qualunque strumento che rispetti la sequenza di chiamata prescritta dallo standard può utilizzare correttamente qualsiasi FMU conforme.

2.3.2 Struttura di un file FMU e modalità operative

Un file FMU è fisicamente un archivio ZIP rinominato con estensione `.fmu` [Mod14]. La struttura interna è standardizzata e prevede i seguenti componenti principali [Mod14, The24]:

- **modelDescription.xml:** è l'unico file obbligatorio. Contiene tutta la descrizione statica del modello: nome, identificatore, versione FMI, GUID di

identificazione univoca, elenco delle variabili scalari con tipo, unità, valore iniziale e direzione (input, output, parametro, variabile di stato), e i flag che dichiarano quali funzionalità opzionali dello standard l'FMU supporta.

- **binaries/**<piattaforma>/: directory che contiene le librerie condivise compilate per ciascuna piattaforma target. La struttura delle sottodirectory segue la convenzione <architettura>-<sistema operativo>, ad esempio `x86_64-linux` o `x86_64-windows`. Un singolo file FMU può contenere binari per piattaforme multiple, garantendo portabilità senza ricompilazione.
- **sources/**: directory opzionale contenente il codice sorgente C del modello. La sua presenza consente la ricompilazione del componente per target non coperti dai binari precompilati — aspetto rilevante per ambienti embedded o architetture non standard.
- **resources/**: directory opzionale per dati accessori richiesti dal modello a tempo di esecuzione, come tabelle di lookup, file di configurazione o mappe.

Il file `modelDescription.xml` costituisce il contratto tra il produttore e il consumatore dell'FMU: definisce in modo dichiarativo e leggibile da macchina tutto ciò che il simulatore deve sapere per utilizzare correttamente il componente, prima ancora di caricare la libreria condivisa. In particolare, l'elenco delle *ScalarVariable* — con i relativi *valueReference*, ovvero gli identificatori numerici usati nelle chiamate `fmi2Get`/`fmi2Set` — costituisce la mappa tra i nomi simbolici delle variabili e le loro rappresentazioni interne.

Dal punto di vista di questo progetto, la struttura descritta pone una sfida concreta: accedere a un file FMU richiede di estrarre l'archivio ZIP, analizzare il file XML, e caricare dinamicamente la libreria condivisa appropriata per la piattaforma corrente. Queste operazioni vengono delegate a una libreria nativa C esistente, che gestisce l'interazione con il file FMU e ne astrae i dettagli di basso livello. Il punto rilevante per le scelte architetturali di questo progetto è che tale libreria espone un'interfaccia C, non accessibile in ambienti JavaScript, ma integrabile direttamente in Kotlin/Native tramite il meccanismo `cinterop` descritto nella sezione precedente — aspetto che verrà ripreso nei capitoli successivi.

Chapter 3

Analisi

Il presente capitolo descrive i requisiti funzionali del sistema e il modello del dominio, con particolare attenzione alle entità coinvolte e ai vincoli che ne regolano il comportamento.

3.1 Requisiti

Di seguito sono elencati i requisiti funzionali del sistema, ovvero le capacità che il sistema deve offrire all'utente o ai componenti che con esso interagiscono.

1. **Caricamento di un file FMU:** Il sistema deve consentire il caricamento di un file nel formato FMU (`.fmu`) tramite interfaccia web. Il file deve essere validato (estensione, formato) e salvato.
2. **Inizializzazione del modello:** Il sistema deve consentire l'inizializzazione del modello FMU precedentemente caricato attraverso l'utilizzo di una libreria nativa, per poi visualizzarne i metadati.
3. **Ispezione dei metadati:** Il sistema deve esporre le informazioni descrittive del modello FMU caricato, tra cui: nome del modello, autore, versione dello standard FMI, tipo di FMU (Co-Simulation o Model Exchange), parametri dell'esperimento di default (istante di inizio, fine e passo), e lista delle variabili con la relativa unità di misura.

4. **Esecuzione della simulazione:** Il sistema deve consentire di avviare una simulazione Co-Simulation sul modello caricato, accettando in input una configurazione (istante di inizio, istante di fine, passo di integrazione, e lista di variabili di output).
5. **Visualizzazione dei risultati:** Il frontend deve presentare i risultati della simulazione in forma grafica, tramite grafici a linea che rappresentino l'andamento temporale delle variabili selezionate.
6. **Selezione delle variabili di output:** L'utente deve poter selezionare un sottoinsieme delle variabili disponibili da includere nell'output della simulazione.
7. **Ricompilazione di FMU con sorgenti (macOS):** Su piattaforma macOS ARM64, il sistema deve essere in grado di ricompilare FMU che includono i file sorgente C, producendo una libreria universale compatibile con le architetture `arm64` e `x86-64`.
8. **Pulizia delle risorse:** Al termine del ciclo di vita del server, il sistema deve liberare le risorse allocate (file temporanei, librerie native) in modo sicuro e deterministico.

3.2 Modello del dominio

Il sistema opera su modelli di simulazione distribuiti nel formato FMU. Il flusso principale prevede che un file FMU venga fornito al sistema, che ne estrae i metadati e li rende disponibili, e che sulla base di questi sia possibile avviare una simulazione e ottenerne i risultati.

Un **modello FMU** è il file che viene condiviso al sistema. All'interno ha diverse componenti, tra cui il file che contiene l'insieme di metadati descrittivi — nome, autore, versione, tipo — e da un insieme di **variabili scalari**, ciascuna con un nome e un'unità di misura. Il tipo del modello determina le operazioni che il sistema può svolgere su di esso: i modelli *Co-Simulation* supportano l'esecuzione di simulazioni, mentre dei modelli *Model Exchange* si possono semplicemente visualizzare i dati.

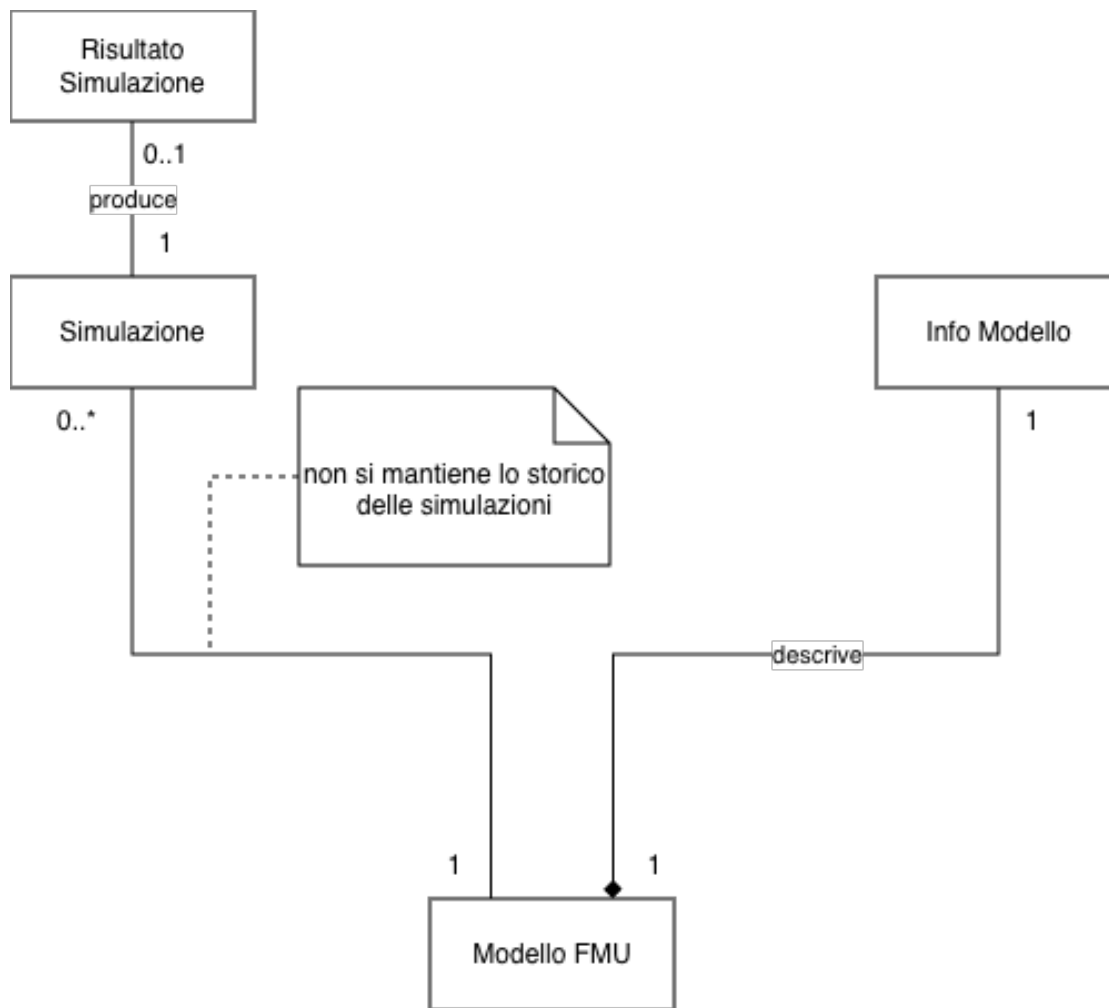


Figure 3.1: Entità dell'applicativo

Una **simulazione** è l'esecuzione di un modello Co-Simulation in un intervallo temporale definito da un istante di inizio, un istante di fine e un passo di integrazione. È possibile specificare un sottoinsieme delle variabili del modello da osservare; se non indicato, vengono registrate tutte le variabili disponibili.

Il **risultato** di una simulazione è la serie temporale prodotta: per ogni istante campionato, il valore assunto da ciascuna variabile osservata. Il risultato mantiene riferimento alla configurazione che lo ha generato.

Il sistema gestisce un solo modello alla volta. Il caricamento di un nuovo file FMU sostituisce il modello precedente e invalida eventuali stati associati a esso.

3.2.1 Vincoli

I vincoli di progetto delimitano lo spazio entro cui il sistema deve essere realizzato.

- **Monolinguaggio:** l'intero sistema deve essere sviluppato in un unico linguaggio di programmazione, senza ricorrere a codice collante in altri linguaggi per l'integrazione con le librerie native — vincolo che, come discusso nel background, esclude le soluzioni full stack tradizionali basate su JavaScript.
- **Compatibilità FMI 2.0:** il sistema deve operare su file FMU conformi allo standard FMI 2.0. Le versioni precedenti o successive dello standard non sono considerate.
- **Portabilità multiplatforma:** il sistema deve essere eseguibile su Linux, macOS e Windows senza modifiche al codice sorgente.
- **Utilizzo di una libreria nativa esistente:** l'interazione con i file FMU deve avvenire tramite una libreria C preesistente che implementa lo standard FMI. Non è previsto lo sviluppo di un'implementazione propria del protocollo.

Chapter 4

Design

4.1 Architettura

Il sistema è strutturato secondo un'architettura client-server a tre livelli, in cui ogni livello è realizzato come modulo indipendente all'interno di un unico progetto multi-modulo. I quattro moduli che compongono il progetto sono `fmilib`, `fmukt`, `backend` e `frontend`, ciascuno con responsabilità ben delimitate e dipendenze unidirezionali.

Il modulo `fmilib` costituisce la base della catena di dipendenze: si occupa della compilazione della libreria nativa *FMILib* e ne espone gli artefatti (header C e librerie condivise) agli strati superiori. Questa scelta isola completamente la toolchain di compilazione dal resto del progetto Kotlin, rendendo il confine tra codice nativo e codice multiplatforma esplicito e controllato.

Il modulo `fmukt` costituisce il cuore della logica di dominio. Dipende da `fmilib` e si occupa di tutto ciò che riguarda il ciclo di vita degli FMU: caricamento del file, parsing dell'XML descrittivo, invocazione della libreria nativa per l'esecuzione della simulazione, e restituzione dei risultati. Il modulo è compilato come libreria Kotlin Multiplatform per le piattaforme native (Linux x64, macOS ARM64 e Windows x64), consentendo di riutilizzare la logica comune tra le piattaforme e di isolare, nei codici sorgente specifici, le parti che richiedono comportamenti differenziati — in particolare la fase di preelaborazione degli FMU su macOS ARM, che richiede la ricompilazione del modello.

Il modulo **backend** implementa un server HTTP REST che espone le funzionalità di **fmu-kt** come API consumabile dal frontend, è anch'esso un modulo Kotlin Multiplatform con target nativi. Il server espone endpoint per il caricamento del file FMU, l'inizializzazione, la lettura dei metadati e l'esecuzione della simulazione.

Il modulo **frontend** realizza l'interfaccia utente web. Il frontend comunica con il backend, permettendo così all'utente di interagire con gli endpoint esposti e di avere una visualizzazione grafica delle informazioni e dei risultati della simulazione.

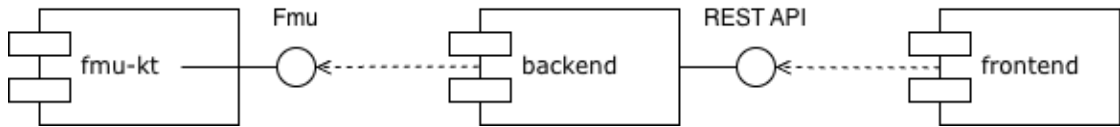


Figure 4.1: Diagramma rappresentante relazione tra i moduli dell'applicativo.

Questa scelta disaccoppia completamente i tre processi: il backend può essere avviato indipendentemente, testato in isolamento tramite client HTTP standard, e in futuro potrebbe essere esposto su rete senza modifiche architetturali. Il frontend non conosce nulla della logica FMU; conosce solo i contratti delle API REST. Mentre il modulo **fmu-kt** funge da libreria indipendente, dunque può essere esportata per essere usata in altri progetti.

Una conseguenza rilevante di questa architettura è la separazione netta tra la parte del sistema che richiede accesso nativo — **fmu-kt** e **backend**, che devono interagire con la libreria C e con il filesystem — e la parte puramente applicativa del frontend, che rimane nel dominio Kotlin puro. Questo ha guidato la decisione di non integrare **fmu-kt** direttamente nel frontend, evitando di contaminare il modulo UI con dipendenze native.

4.2 Design di dettaglio

Le sezioni seguenti approfondiscono le scelte di design dei tre componenti principali del sistema, seguendo lo stesso ordine delle dipendenze architetturali descritte in precedenza: si parte dall'interfacciamento con le librerie native, che costituisce il fondamento su cui si appoggiano gli strati superiori, per risalire al backend — che

orchestra la logica di dominio ed espone i servizi verso l'esterno — fino al frontend, responsabile della presentazione e dell'interazione con l'utente.

Per ciascun componente vengono discusse le scelte progettuali significative, con particolare attenzione ai confini tra codice condiviso e codice specifico per piattaforma, e alle motivazioni che hanno guidato la separazione delle responsabilità.

4.2.1 Interfacciamento con librerie native

Il modulo `fmu-kt` è il componente del sistema responsabile di tutta l'interazione con la libreria nativa FMILib. Il suo design è orientato ad astrarre la complessità di basso livello esposta dalla libreria C, presentando agli strati superiori un'interfaccia di alto livello che non richiede alcuna conoscenza dei dettagli nativi.

Il punto di accesso esterno al modulo è `FmuService`, un'interfaccia che espone le operazioni disponibili sugli FMU: caricamento, lettura dei metadati e avvio della simulazione. Questa scelta consente al backend di dipendere esclusivamente dal contratto dell'interfaccia, rendendo il modulo sostituibile e facilmente testabile tramite implementazioni fittizie. Ulteriori funzionalità potranno essere introdotte in futuro aggiungendo nuovi metodi all'interfaccia e le relative implementazioni, senza alterare la struttura generale del modulo.

L'implementazione concreta del servizio, denominata `DefaultFmuService` delega la gestione del modello a `FmuManager`, un componente che orchestra l'intero ciclo di vita di un FMU attraverso tre manager specializzati, ciascuno con una responsabilità ben delimitata:

- **FmuLifecycleManager**: si occupa dell'apertura e della chiusura dell'FMU, gestendo l'allocazione e il rilascio delle risorse associate.
- **InfoManagerFmu**: estrae i metadati del modello — nome, autore, versione, variabili disponibili e parametri dell'esperimento predefinito — interrogando direttamente le strutture dati native.
- **SimulationManager**: gestisce la configurazione e l'esecuzione della simulazione raccogliendo i risultati e restituendoli in un formato indipendente dalla libreria nativa.

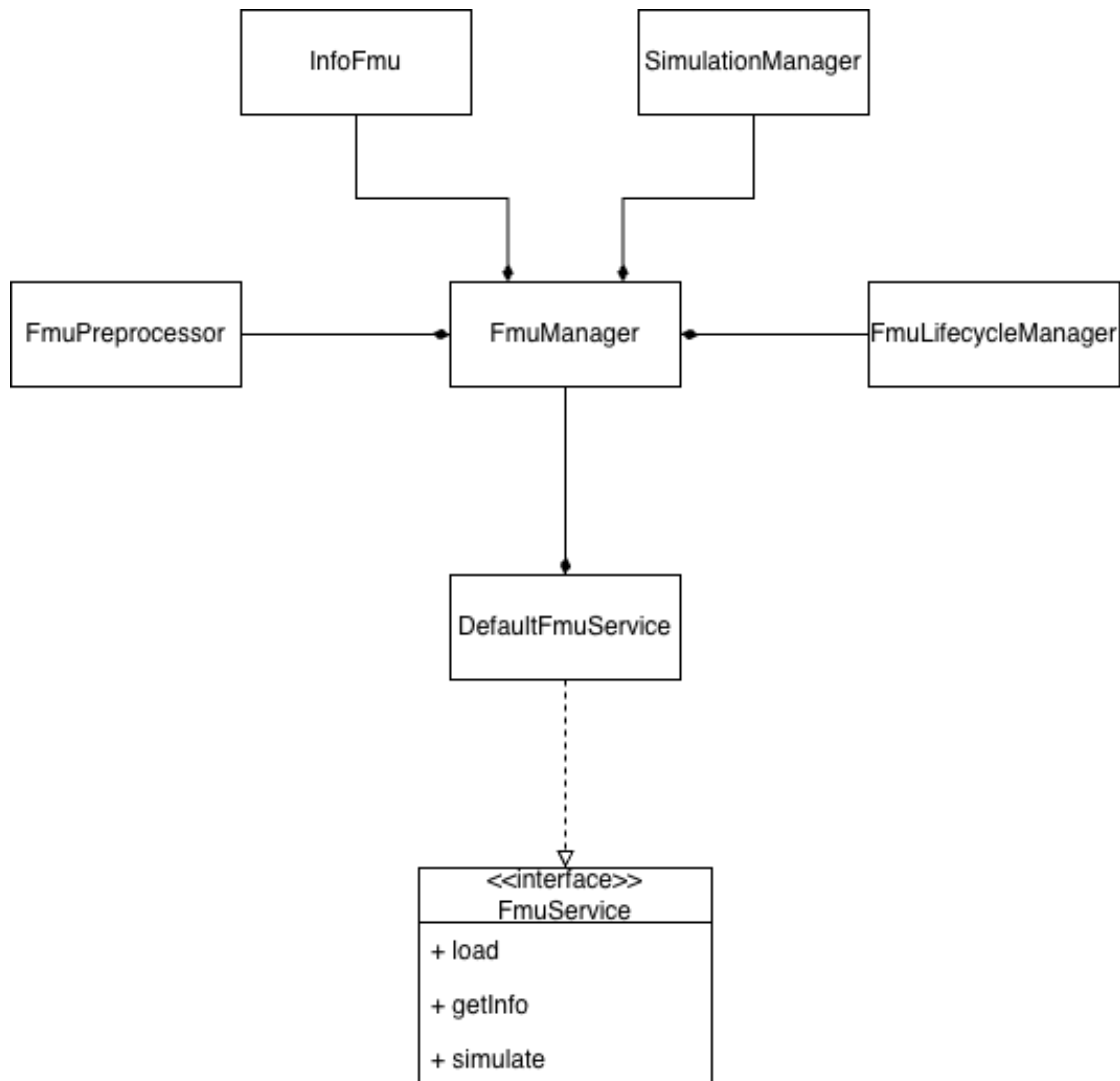


Figure 4.2: Design del modulo fmu-kt

Questi tre manager sono gli unici componenti del sistema che interagiscono direttamente con FMILib. Concentrare questo confine in un insieme ristretto e ben identificato di classi rende il sistema più manutenibile: eventuali cambiamenti nell'API nativa o l'adozione di una libreria FMI alternativa richiederebbero modifiche localizzate nei manager, senza propagarsi al resto dell'architettura.

Prima che un FMU possa essere caricato, è necessaria una fase di preparazione del file che ne garantisce la compatibilità con la piattaforma corrente. Questa responsabilità è affidata al `FmuPreprocessor`, un componente separato la cui implementazione varia a seconda della piattaforma target. La sua interfaccia è definita nel source set comune, mentre le implementazioni specifiche risiedono nei source set di piattaforma, seguendo il meccanismo `expect/actual` di Kotlin Multiplatform. In questo modo la logica di preparazione dell'FMU rimane completamente trasparente agli strati superiori.

4.2.2 Backend multiplatforma

Il backend è strutturato in modo da separare nettamente la definizione delle rotte HTTP dalla logica applicativa che ne gestisce le richieste. Il componente centrale di questa separazione è `RequestHandler`, che raccoglie tutta la logica di gestione delle richieste e viene iniettato nel layer di routing al momento dell'avvio del server. In questo modo le rotte rimangono dichiarative e di facile lettura, limitandosi a descrivere la mappatura tra endpoint e operazioni, mentre tutta la logica risiede in un unico posto ben identificato. Aggiungere o modificare un endpoint richiede quindi interventi localizzati e indipendenti tra loro.

`RequestHandler` è a sua volta una composizione di due componenti, ciascuno con una responsabilità distinta:

- **ResourceManagerService**: è l'interfaccia che definisce le operazioni di gestione del filesystem locale, in particolare del salvataggio del file FMU ricevuto dal client e della restituzione dei percorsi necessari al caricamento del modello. Isolare questa responsabilità in un componente dedicato rende esplicito il confine tra la gestione delle risorse persistenti e la logica di interazione con il modello, e semplifica la sostituzione della strategia di memorizzazione, ad esempio in fase di test.

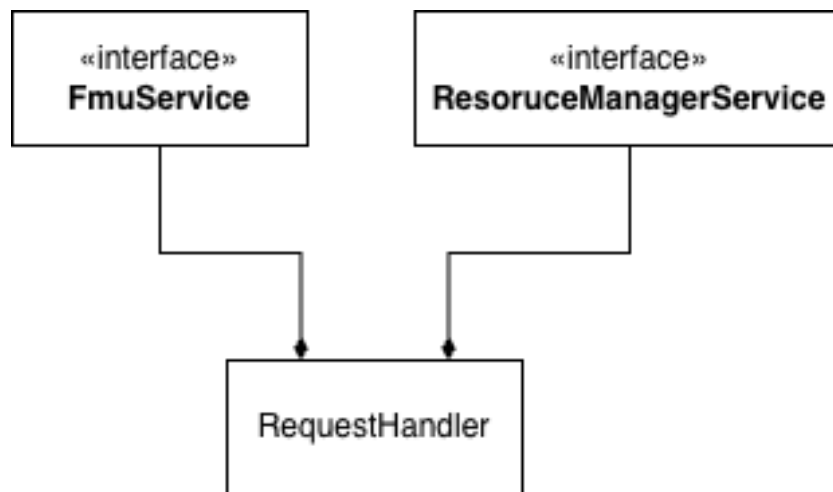


Figure 4.3: Design del modulo backend

- **FmuService**: è l'interfaccia del modulo `fmu-kt` descritta nella sezione precedente. Il backend non dipende da nessun dettaglio interno di quel modulo: conosce solo le operazioni che il servizio espone, mantenendo il disaccoppiamento tra i due moduli.

Questa composizione rispecchia il flusso naturale di una richiesta: quando il client avvia una simulazione, `RequestHandler` gestisce il caricamento del file, che viene salvato dal `ResourceManagerService` e successivamente fornito a `FmuService` per il caricamento e l'esecuzione del modello. Le due responsabilità — dove si trova il file e cosa farne — restano separate e intercambiabili indipendentemente. La scelta far dipendere `RequestHandler` da un'interfaccia piuttosto che a una classe, per quanto riguarda la gestione del filesystem, è motivata dalla volontà di astrarre l'organizzazione interna dei file, permettendo così di realizzare un sistema di salvataggio organizzato in modo diversa senza intaccare in alcun modo il `RequestHandler`.

4.2.3 Frontend multiplatforma

Il frontend è organizzato attorno a due responsabilità principali, riflesse nella struttura del modulo: la presentazione dell'interfaccia utente e la comunicazione con il backend.

La navigazione tra le diverse viste dell'applicazione è gestita attraverso il concetto di *screen*, ognuno dei quali rappresenta una schermata autonoma con una funzione ben precisa. Sono presenti una schermata iniziale (**HomeScreen**), una per la visualizzazione dei metadati del modello caricato (**InfoScreen**), una per la configurazione e l'avvio della simulazione (**SimulationScreen**) e una per la visualizzazione grafica dei risultati (**ChartScreen**). Questa suddivisione rende il codice dell'interfaccia facilmente navigabile e consente di aggiungere nuove schermate senza interferire con quelle esistenti.

La comunicazione con il backend è invece centralizzata nel modulo **client**, che espone le funzioni necessarie per interagire con ciascun endpoint REST. Concentrare questa logica in un unico punto evita la duplicazione del codice di comunicazione tra le schermate e isola i dettagli del protocollo HTTP dal codice dell'interfaccia utente. Le schermate si limitano quindi a invocare le funzioni del client e a reagire ai risultati, senza conoscere nulla della comunicazione sottostante.

Chapter 5

Implementazione

Questo capitolo descrive le scelte implementative principali del sistema, seguendo un percorso che va dagli strati più bassi verso quelli più alti, in modo da costruire progressivamente il contesto necessario alla comprensione di ciascuna parte.

Si parte dal binding tra Kotlin e la libreria nativa FMILib, illustrando come il meccanismo di interoperabilità con il codice C sia stato configurato e come le opzioni del linker siano state adattate per ciascuna piattaforma target. Su questa base si descrive poi la gestione del ciclo di vita dell'FMU, ovvero come il sistema allochi il contesto FMI, effettui il parsing dell'XML descrittivo e carichi il binario nativo. Si procede quindi con l'estrazione dei metadati e la gestione delle variabili, spiegando come le informazioni del modello vengano interrogate tramite FMILib e restituite in un formato indipendente dalla libreria. La sezione sull'esecuzione della simulazione descrive invece il loop di avanzamento temporale, la risoluzione dei value reference e la raccolta dei risultati a ogni passo.

Una sezione dedicata tratta la ricompilazione degli FMU su macOS ARM64, un caso particolare richiesto dall'architettura Apple Silicon, in cui i binari distribuiti negli FMU non sono compatibili con la piattaforma e devono essere ricompilati a partire dai sorgenti C inclusi nel pacchetto. Il capitolo si chiude con una panoramica degli strumenti di processo adottati: la configurazione Gradle multipiattaforma, la pipeline di CI/CD e il workflow di build e test su Linux, macOS e Windows.

5.1 Binding C/Kotlin e configurazione del linker per piattaforma

L'interoperabilità tra Kotlin e la libreria nativa FMILib è realizzata tramite il meccanismo di C interop di Kotlin/Native, che genera automaticamente binding Kotlin a partire dagli header C della libreria. La configurazione del binding avviene tramite un file `.def` in cui vengono dichiarati gli header da includere e le opzioni di compilazione e linking necessarie, tra cui il percorso alla directory degli header e il nome della libreria condivisa da collegare. A partire da questa dichiarazione, il compilatore produce un modulo Kotlin che espone le funzioni e i tipi C come API direttamente utilizzabili dal codice Kotlin, boilerplate manuale minimo rappresentato dalla scrittura del file `.def`.

La libreria FMILib non è disponibile come artefatto precompilato su repository pubblici, ma viene compilata a partire dai sorgenti tramite CMake. La compilazione è gestita dal modulo `fmilib`, che si occupa di invocare CMake, eseguire l'installazione degli artefatti e renderli disponibili al resto del progetto attraverso una configurazione e un plugin Gradle dedicato. Il modulo `fm-kt` dichiara una dipendenza su questi artefatti e li referencia nelle opzioni di linker del target nativo, includendo sia la directory degli header sia quella delle librerie condivise prodotte dalla compilazione.

Poiché il sistema deve essere compilato su tre piattaforme distinte — Linux x64, macOS ARM64 e Windows x64 — le opzioni di linker sono configurate per piattaforma, in quanto i percorsi degli artefatti e le convenzioni del linker differiscono tra i sistemi operativi. Su Windows, ad esempio, la libreria condivisa è un file `.dll` che deve essere copiato nella stessa directory dell'eseguibile prodotto; su Linux e macOS si tratta invece di file `.so` e `.dylib` referenziabili tramite `rpath`.

Su Linux si è reso necessario un ulteriore workaround legato alla toolchain interna di Kotlin/Native. Il compilatore Konan include un proprio sysroot con una versione di glibc datata (2.19, kernel 4.9), pensata per massimizzare la compatibilità tra distribuzioni. Quando la libreria da collegare è stata compilata con una versione più recente di glibc — come accade negli ambienti CI moderni — il linker di Konan non riesce a risolvere alcuni simboli di sistema presenti nella

libreria condivisa di FMILib. Per ovviare a questo problema si è fatto ricorso alle opzioni `--allow-shlib-undefined` e `--unresolved-symbols=ignore-all`, che istruiscono il linker a non fallire in presenza di simboli non risolti nelle librerie condivise, delegandone la risoluzione al linker dinamico del sistema operativo a tempo di esecuzione. Questa è la soluzione consolidata nella comunità Kotlin/Native per questo tipo di incompatibilità, accettabile in quanto i simboli in questione sono garantiti essere presenti in qualsiasi distribuzione Linux moderna su cui il binario verrà eseguito.

5.2 Gestione del ciclo di vita dell'FMU

5.3 Estrazione dei metadati e gestione delle variabili

5.4 Esecuzione della simulazione

5.5 Ricompilazione degli FMU su macOS ARM64

5.6 Strumenti di processo: Gradle, CI/CD e pipeline multiplatforma

Chapter 6

Valutazione

6.1 Test unitari e di integrazione

6.2 Copertura multiplatforma nella CI

Chapter 7

Conclusioni e Sviluppi Futuri

7.1 Limitazioni

7.2 Sviluppi futuri

7.3 Conclusioni

Bibliography

- [AB⁺20] Marat Akhin, Mikhail Belyaev, et al. Kotlin language specification: Kotlin/core. Technical report, JetBrains / JetBrains Research, 2020.
- [Jet24a] JetBrains. Expected and actual declarations. <https://kotlinlang.org/docs/multiplatform-expect-actual.html>, 2024.
- [Jet24b] JetBrains. Interoperability with c. <https://kotlinlang.org/docs/native-c-interop.html>, 2024.
- [Jet24c] JetBrains. Kotlin multiplatform. <https://kotlinlang.org/docs/multiplatform.html>, 2024.
- [Mod14] Modelica Association. Functional mock-up interface for model exchange and co-simulation, version 2.0. Technical report, Modelica Association Project, 2014.
- [Mod24] Modelica Association. Functional mock-up interface. <https://fmi-standard.org>, 2024.
- [Nod25] Node.js. Node-API. <https://nodejs.org/api/n-api.html#node-api>, 2025. Accessed: 2026-06-10.
- [Sta25] Stack Overflow. 2025 stack overflow developer survey. <https://survey.stackoverflow.co/2025/>, 2025. Accessed: 10 June 2026.
- [T⁺21] Casper Thule et al. RMQFMU: Bridging the real world with co-simulation. Technical report, arXiv, 2021.

BIBLIOGRAPHY

- [The24] The MathWorks, Inc. Functional mockup interface concepts. <https://www.mathworks.com/help/fmuexport/gs/fmuexport-concepts.html>, 2024.